

Runbook · operación de la v2 (FastAPI + Django)

Qué corre

| Proceso | Comando | Puerto | |---|---|---| | API | backend-fastapi/.venv/bin/uvicorn app.main:app
--host 127.0.0.1 --port 5002 | 5002 | | Front | frontend-django/.venv/bin/python
manage.py runserver 0.0.0.0:8001 | 8001 |

El front no llama a la API desde el navegador: el JWT vive en la sesión de Django y las vistas hacen de proxy (frontend-django/atenciones/api.py). Por eso la API puede quedar escuchando solo en localhost.

Los dos necesitan sus variables de entorno cargadas (set -a && source .env && set +a).

Variables de entorno

backend-fastapi/.env

| Variable | Para qué | |---|---| | DATABASE_URL | Postgres de la v2 | | JWT_SECRET | firma los tokens de la API | | SEED_PASSWORD | solo para scripts/seed.py |

frontend-django/.env

| Variable | Para qué | |---|---| | DJANGO_SECRET_KEY | firma las sesiones y el CSRF | | DJANGO_DEBUG | 1 en dev, 0 en producción | | DJANGO_HTTPS | 1 solo detrás de un proxy con HTTPS | | FASTAPI_URL | dónde está la API | | DJANGO_ALLOWED_HOSTS | hosts permitidos, separados por coma (sin *) |

Generar los secretos (nunca versionarlos):

```
python -c "import secrets; print(secrets.token_urlsafe(64))"
```

- DJANGO_SECRET_KEY: con esto se firman las sesiones. Si es corta o predecible, se puede falsificar la cookie de un jefe y entrar como él. Mínimo 50 caracteres al azar.
- JWT_SECRET: igual, pero para los tokens de la API.

Base de datos

El esquema es reproducible con alembic. Desde backend-fastapi/:

```
alembic upgrade head # crea o actualiza el  
esquema  
python scripts/seed.py # usuarios + catálogo +  
atenciones
```

Para ponerse al día con lo que producción siga registrando (ver D1 pendientes-spec.md 

```
python scripts/extraer_septiembre.py # refresca el CSV desde  
prod  
python scripts/actualizar_atenciones.py # suma solo lo que falta
```

Los dos son idempotentes: se pueden correr las veces que haga falta.

Ojo: seed.py re-hashea la contraseña de todos los usuarios. Para sumar atenciones no lo uses: para eso está actualizar_atenciones.py.

Levantar la base de producción (promoción)

La v2 arranca con **septiembre como mes 1**: base propia, catálogo propio, y las 276 atenciones de septiembre. Desde backend-fastapi/, con el esquema puesto:

```
python scripts/promover_a_produccion.py <DATABASE_URL_destino>
```

Hace cuatro cosas y se niega a correr si el destino ya tiene usuarios o atenciones:

| Qué | De dónde sale | -- | -- | Catálogo (3 sectores, 4 grupos, 53 áreas) | mapeo-areas-dedup.csv | | **Usuarios** (9, con su contraseña) | dev1, copiando el hash bcrypt tal cual | | Atenciones de septiembre (276) | atenciones_septiembre.csv | | Horarios | la base de origen, remapeando por email |

Los hashes se copian porque v1 y v2 usan bcrypt: **nadie tiene que cambiar su contraseña** Verificado comparando los 9 contra v1.

La base destino se crea con `alembic upgrade head` (el script no la crea) y conviene probarla antes en una base descartable, no encima de la de trabajo.

Correr los tests

Los del front crean su propia base descartable, así que no tocan nada.

Los del back **escriben en la base a la que apunte DATABASE_URL**, y desde el corte esa es la de producción. Por eso tests/conftest.py **se niega a correr** si el nombre de la base no termina en `_test`. Hay que usar una copia:

```
# una copia de produccion, con las contraseñas del seed para poder
loguearse
docker exec soporte-postgres-dev psql -U soporte -d postgres -c
"CREATE DATABASE soporte_test;"
docker exec soporte-postgres-dev sh -c "pg_dump -U soporte -d
soporte -Fc | pg_restore -U soporte -d soporte_test"
cd backend-fastapi && DATABASE_URL=<...>/soporte_test python
scripts/seed.py
# y los tests, contra esa copia
cd backend-fastapi && DATABASE_URL=<...>/soporte_test python -m
pytest -q
cd frontend-django && python manage.py test tests
```

Los CSV de datos no están versionados

atenciones_septiembre.csv, mapeo-areas.csv y mapeo-areas-dedup.csv están en el .gitignore: son datos, no código. Consecuencia para un despliegue nuevo **el repo por sí solo no alcanza para sembrar la base**. Cómo se consigue cada uno:

- **atenciones_septiembre.csv**: se regenera de prod con scripts/extraer_septiembre.py.
- **mapeo-areas.csv** y **mapeo-areas-dedup.csv**: son la **definición del catálogo**, no se pueden regenerar (en prod el catálogo está vacío). Hay que copiarlos junto con el despliegue, o sacar el catálogo de una base que ya lo tenga (`GET /api/jerarquia/arbol`).

Horarios

La carga es **manual y mensual**: una fila por usuario y día de la semana, desde `horarios`. Para lo repetitivo está "copiar mes anterior". Los domingos no tienen turno para nadie, y eso es por diseño: el estado de ese día es "fuera de turno".

Backups

/home/mattias/backups/soporte/, contenedor soporte-pgbackup. Vale revisar que el dump más reciente pese más de 0: el pipeline ya generó 1.1M de archivos vacíos una vez, por ~~un~~ que se saltaba el sleep. **No hay alerta automática todavía.**

Restaurar para verificar (en una base de prueba, nunca encima de la de trabajo):

```
docker exec -i soporte-postgres-dev psql -U soporte -d
<base_de_prueba> < dump.sql
```

Antes de exponerlo a los usuarios

- 1 **DJANGO_DEBUG=0**. Con Debug encendido, cualquier error le muestra código y configuración a quien lo provoque.
- 2 **DJANGO_SECRET_KEY** y **JWT_SECRET** **fuertes** y propias de producción.
- 3 **HTTPS**, con un proxy que mande X-Forwarded-Proto: https y **DJANGO_HTTPS=1** (activa cookies seguras, redirección a HTTPS y HSTS) Si el proxy no manda ese header, la redirección entra en **bucle infinito**.
- 4 **DJANGO_ALLOWED_HOSTS** con el host real, sin *.
- 5 **Una contraseña por usuario** Hoy el seed las deja todas iguales. Los hashes de v1 son bcrypt (\$2b\$) y son compatibles: se pueden copiar tal cual y nadie tiene que cambiar su contraseña.
- 6 **Verificar** con, desde frontend-django/:

```
python manage.py check --deploy
```

Queda **un** warning a propósito: **SECURE_HSTS_PRELOAD**. Es para entrar a la lista de precarga de los navegadores, no aplica a un sistema interno, y es difícil de revertir.

Dónde vive cada cosa (ojo con los nombres)

```
|| Contenedor | Base | Se llega por | ---|---|---|---| v2 (producción desde el 2026-09-23)
soporte-postgres-dev | soporte | host : 5433 || v1 (apagada, queda como evidencia) |
soporte-postgres | soporte | sin puerto al host: docker exec |
```

Las dos bases se llaman soporte y el contenedor de producción se llama dev. Es confuso y es deuda del arranque: se ordena cuando la v2 tenga su contenedor propio.

La v1 quedó apagada pero entera docker start soporte-backend soporte-frontend la vuelve a levantar en :3002. La base y el backup de la v1 nunca se tocaron. Su dump final está en ~/backups/soporte/prod_v1_final_*.dump.

Trabajar un cambio sin romper producción

Mientras no haya contenedores, la regla es **una máquina, dos instancias** producción o se toca, el cambio se prueba en una copia.

```
# 1. una copia de la base de producción, para trabajar sin riesgo
docker exec soporte-postgres-dev pg_dump -U soporte -d soporte -Fc
> /tmp/prod.dump
docker exec soporte-postgres-dev psql -U soporte -d postgres -c
"CREATE DATABASE soporte_dev;"
docker exec -i soporte-postgres-dev pg_restore -U soporte -d
soporte_dev < /tmp/prod.dump
# 2. una rama por cambio, y esa base en la instancia de trabajo
```

```
git checkout -b dev-lo-que-sea
# front en :8011 y API en :5012, apuntando a soporte_dev
```

Para publicar:

```
git checkout python-experiment && git merge dev-lo-que-sea
alembic upgrade head # migraciones aditivas,
nunca destructivas
bash scripts/arrancar_produccion.sh # o: systemctl restart
soporte-api soporte-web
```

Tres reglas que no se negocian:

- 1 **Un dump antes de cada publicación** Si algo sale mal, se restaura y listo.
- 2 **Nunca editar el código de la instancia que está corriendo** Todo entra por git, aunque sea un cambio de una línea. Lo que corre en producción es lo que está en `main`, no lo que alguien tocó a mano.
- 3 **Las migraciones no borran datos.** Agregar columnas sí; renombrar o eliminar requiere una migración en dos pasos (agregar, migrar, y recién después borrar).

Arranque y control de los servicios

Los dos procesos los maneja `systemd` (unidades en `deploy/systemd/`, instaladas el 2026-09-23). Antes se levantaban a mano con `scripts/arrancar_produccion.sh`, que sigue sirviendo para un arranque de emergencia o para reinstalar.

```
sudo systemctl status soporte-api soporte-web # estan vivos?
sudo systemctl restart soporte-web # despues de
publicar un cambio
journalctl -u soporte-api -n 50 # o tail de logs/
```

Están `enabled` (arrancan solas al reiniciar la máquina) y con `Restart=always`: si un proceso se cae, vuelve solo. Probado matando los dos con `SIGKILL`.

Para instalarlas en otra máquina:

```
sudo cp deploy/systemd/*.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now soporte-api soporte-web
```

Lo que todavía no está

- El sidebar de `/auxiliares/`: es un cartel de "próximamente".
- **Reportes**: la página de v1 era una maqueta sin `fetch`. Falta decidir si es un feature real y de qué tipo.
- Alertas si el backup vuelve a fallar.